Project 3
CS448 Computer Security
Project Report

Jena Blose and Trinity Thompson

Password Generator and Secure Vault System

Introduction

Passwords remain the most common authentication method used for online accounts and digital services. However, weak passwords are one of the primary causes of account compromise. Password generators and password managers help reduce this risk by creating strong passwords and securely storing them so users do not need to remember them. The goal of this project was to design and implement a password generator and secure storage system inspired by modern password management tools such as NordPass. The application allows users to generate strong passwords or passphrases, evaluate their strength, and securely store credentials for later retrieval.

The system was designed with both usability and security in mind. Users can generate traditional character based passwords or passphrases composed of dictionary words. The system also includes encrypted storage for credentials, a demonstration of autofill functionality, and validation and storage for credit card information. The project combines a Python backend with a web based interface, demonstrating how secure systems can integrate cryptographic operations, randomness, and user interface design.

System Architecture

The application uses a client server architecture. The backend is implemented in Python using the Flask framework, which provides a lightweight web server and API endpoints. The frontend interface is implemented using HTML, CSS, and JavaScript. The frontend communicates with the backend through HTTP requests to generate passwords, store credentials, and retrieve stored data.

The backend contains several modules responsible for different parts of the system. The password generation module handles the creation of passwords and passphrases using secure randomness. The password evaluation module analyzes password strength and calculates entropy. The vault module securely stores sensitive information using encryption. A card validation module verifies credit card numbers, expiration dates, and security codes before storing them. The Flask server coordinates these components and exposes endpoints that allow the frontend interface to interact with the system. This separation of responsibilities makes the code easier to maintain and demonstrates modular software design.

Password Generation Design

The system supports two different password generation modes. The first mode generates random passwords composed of characters. In this mode the user can select whether to

include uppercase letters, lowercase letters, numbers, and symbols. The user can also choose to avoid ambiguous characters such as the number zero and the letter O, which are often confused with one another. The password generator guarantees that at least one character from each selected category is included in the final password. This ensures that the password meets the user's requirements while still remaining random.

The second generation mode produces passphrases. Passphrases are created by randomly selecting words from a dictionary file and joining them with a separator chosen by the user. Examples of separators include hyphens, underscores, periods, commas, or spaces. The user may also enable options to capitalize the words or apply digit substitution, which replaces certain letters with numbers. This approach produces passwords that are easier to remember while still maintaining strong security because of the large number of possible word combinations.

Randomness Source

The security of a password generator depends heavily on the randomness used to create passwords. In this project, the Python secrets module was used to generate random values. The secrets module is specifically designed for cryptographic applications and is considered more secure than the standard random module for generating passwords or tokens.

The secrets module uses the operating system's secure random number generator, which collects entropy from the system and provides unpredictable random values. This ensures that the generated passwords cannot be predicted by attackers. Each character or word used in password generation is selected using the secrets.choice function, which randomly selects an element from a sequence in a cryptographically secure manner. By relying on secure randomness from the operating system, the generator produces passwords with high unpredictability and resistance to guessing attacks.

Password Strength Evaluation

The project also includes a password strength evaluation system that estimates the security of generated passwords. Strength evaluation combines entropy calculation with rule based classification similar to those used in commercial password managers. Entropy measures how unpredictable a password is and is calculated using the length of the password and the size of the character set used.

Higher entropy indicates that more possible combinations exist, which makes brute force attacks more difficult. In addition to entropy calculations, rule based checks examine characteristics such as password length and the number of character categories used. These rules categorize passwords into three levels which are weak, moderate, and strong.

This classification provides a clear and simple indicator for users while still relying on underlying security principles.

Secure Vault Design

A major component of the project is the secure storage of credentials. When users save passwords or credit card information, the system encrypts the sensitive data before writing it to storage. Encryption is implemented using the Fernet symmetric encryption scheme from the Python cryptography library. Fernet provides authenticated encryption, meaning that stored data cannot be read or modified without the correct encryption key.

The encryption key is generated automatically the first time the program runs and is stored locally in a key file. When a password or credit card is saved, the program encrypts the sensitive fields and stores them in a JSON file. When the data is retrieved, it is decrypted so it can be displayed in the user interface. This approach protects stored information from unauthorized access and demonstrates how password managers protect sensitive user data.

Credit Card Validation

The project also includes a feature for storing credit card information securely. Before a card can be saved, the program performs several validation checks. The card number is first checked using the Luhn algorithm, which is a widely used checksum algorithm designed to detect invalid credit card numbers. The system also determines the type of card based on known number prefixes associated with major providers such as Visa, Mastercard, American Express, and Discover.

The expiration date is validated to ensure that the format is correct and that the card has not expired. In addition, the program checks the CVC security code to ensure that it contains the correct number of digits for the detected card type. Only if all of these validation steps pass will the card information be encrypted and stored. These validation procedures demonstrate how real systems verify payment information before accepting it.

Comparison to Existing Tools

This project was inspired by commercial password managers such as NordPass. Many design ideas were based on features that appear in these professional tools. For example, the project includes options for generating complex passwords, generating passphrases from dictionary words, evaluating password strength, and securely storing credentials. The user interface also mimics the layout of a password manager with categories, tools, and a password vault.

Although the project demonstrates many similar concepts, commercial password managers include additional features such as browser extensions, secure cloud synchronization, biometric authentication, and multi device support. Those systems also use hardened security architectures and extensive testing to protect user data. The project created here focuses on demonstrating the underlying concepts rather than implementing every advanced feature.

Conclusion

This project demonstrates the fundamental principles behind secure password generation and storage systems. By combining secure randomness, encryption, validation algorithms, and a web interface, the system provides a simplified but functional password manager. The application allows users to generate strong passwords, evaluate their strength, and store them securely using encryption.

The project highlights several important cybersecurity concepts including cryptographically secure randomness, entropy based password evaluation, encrypted storage of sensitive information, and validation of financial data. While the system is simplified compared to commercial products, it effectively illustrates the techniques used by real password managers and provides a strong foundation for future development.

AI Usage Statement

Artificial intelligence tools were used during the development of this project to assist with debugging, understanding programming concepts, and improving code structure. AI assistance was primarily used for explaining algorithms, suggesting improvements to program organization, and helping resolve technical errors encountered during development.

All design decisions, implementation choices, and integration of components were completed by the student. The final code was reviewed and tested by the student to ensure correctness and understanding of the implemented functionality.

AI tools were used as a learning aid rather than a replacement for independent development.